Decision Trees

From "What should I ask?" to ID3, C4.5, and CART

Dr. Nikhilanand Arya

IIIT Allahabad

An interactive way to understand decision tree induction

Today's Question

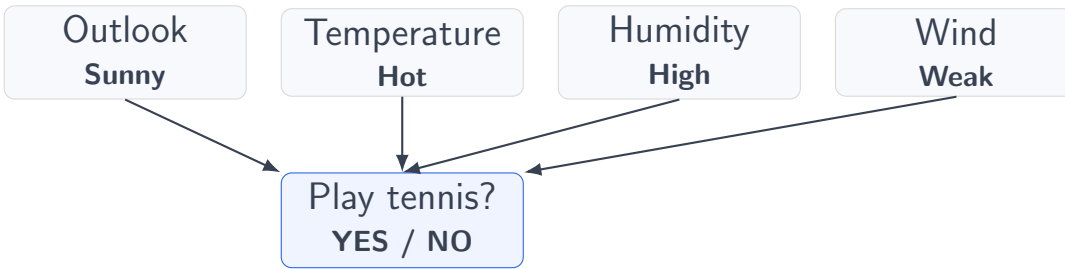
How does a machine decide which question to ask first?



Think

Today we will discover the answer rather than start with the formula.

Warm-up: Should We Play Tennis?



Challenge

You are the ML algorithm. If you can ask only ONE question first, which feature would you choose?

A. Outlook B. Temperature C. Humidity D. Wind

The Play Tennis Dataset

Outlook	Temp	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

Which Group Is Easier to Predict?

YES YES YES YES	NO NO NO NO	YES NO YES NO
Easy	**Easy**	**Uncertain**

Pure group $\longrightarrow$ easy prediction

Mixed group $\longrightarrow$ uncertain prediction

Key Insight

A decision tree needs a numerical way to measure this uncertainty.

Entropy = Uncertainty



$$H(Y) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

Entropy = a number that tells us how uncertain a node is.

- Higher entropy $\rightarrow$ more uncertainty
- Lower entropy $\rightarrow$ less uncertainty
- Entropy $= 0 \rightarrow$ pure node

Entropy of Play Tennis

14 examples

9 Yes 5 No

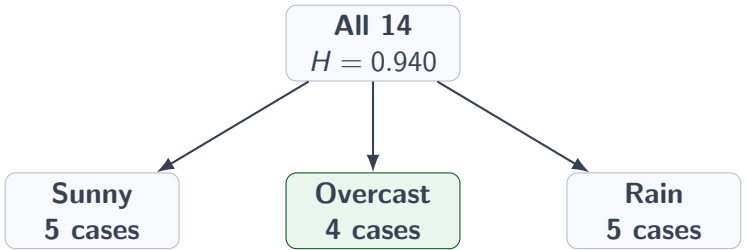
$$H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14}$$

Result

$$H(S) = 0.940$$

This is our **starting uncertainty**.

What Happens If We Ask: "Outlook?"



Challenge

Which split leaves us with the **least confusion** overall?

We need a weighted entropy after the split.

Weighted Entropy: One Calculation

For Outlook:

$$Info_{Outlook}(S) = \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971)$$

$$\boxed{Info_{Outlook}(S) = 0.694}$$

Before split: **0.940**

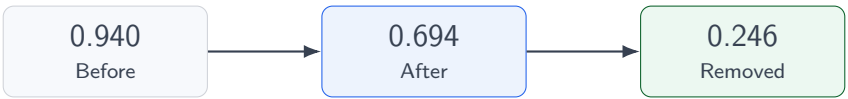
After split: **0.694**

Key Insight

The split reduced uncertainty. How much did it reduce it?

Information Gain = Uncertainty Removed

$$Gain(A) = Info(S) - Info_A(S)$$



Meaning

A good question removes a lot of uncertainty.

Which Question Wins?

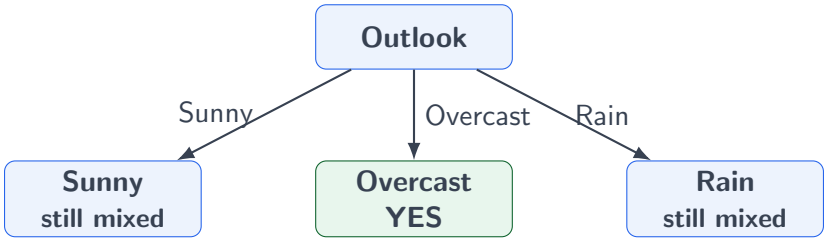
Question	Weighted Entropy	Gain
Outlook	0.694	**0.246** ⋆
Humidity	0.789	0.151
Wind	0.892	0.048
Temperature	0.911	0.029



Decision

Highest Information Gain $\Rightarrow$ **Outlook becomes the root.**

Build the Tree: Round 1



Think

Overcast is pure. Do we need another question for that branch?

Build the Tree: Sunny Branch

Sunny subset:

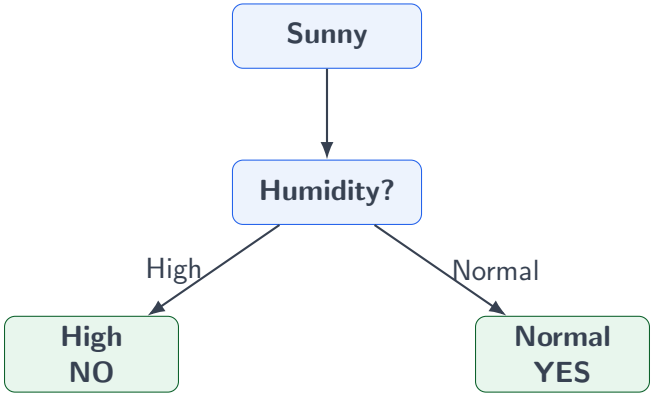
$$2\ \mathit{Yes}, \quad 3\ \mathit{No}, \quad H = 0.971$$

Remaining attribute	Gain
Temperature	0.571
Humidity	**0.971** $\star$
Wind	0.020

Decision

Sunny $\rightarrow$ split on **Humidity**

Sunny Becomes Pure



Both leaves are now **pure**.

Build the Tree: Rain Branch

Rain subset:

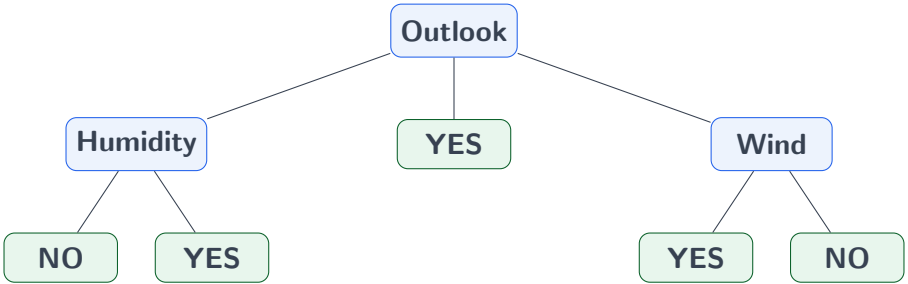
$$3\ \textit{Yes}, \quad 2\ \textit{No}, \quad H = 0.971$$

Remaining attribute	Gain
Temperature	0.020
Humidity	0.020
Wind	**0.971** $\star$

Decision

Rain $\rightarrow$ split on **Wind**

The Final ID3 Tree



Question → Answer → Next question

But Wait... Can Information Gain Be Fooled?

Suppose our dataset contains:

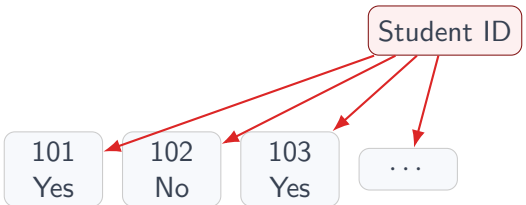
Student ID
101, 102, 103, 104, 105, ...

Challenge

Every ID is unique. What happens if we split using Student ID?

Each record gets its own branch $\Rightarrow$ each branch is pure.

The Student ID Trap



Problem

Perfectly pure training groups $\neq$ useful feature.

This motivates C4.5.

C4.5: A Better Question

Don't just ask:

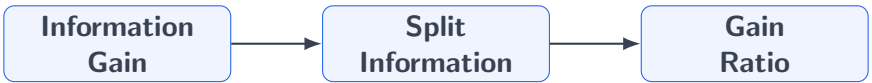
"How much did uncertainty decrease?"

Also ask:

"How complicated was the split?"

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo(A)}$$

C4.5 in One Picture



Reward useful separation
but account for split complexity.

C4.5: What Else Changes?

	ID3	C4.5
Splitting measure	Information Gain	Gain Ratio
Continuous attributes	No	Yes
Missing values	Limited	Yes
Pruning	No	Yes

C4.5 is essentially the **next generation** of the ID3 idea.

Continuous Features: Where Should We Split?

Suppose the feature is **Age**:

15 18 21 22 24 25 29 31 $\cdots$

Candidate thresholds:

16.5, 19.5, 21.5, 23, 24.5, 27, 30, $\ldots$

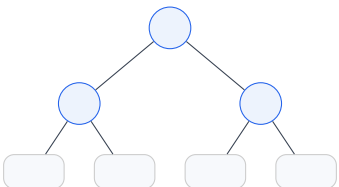
Challenge

Try several thresholds. Which one gives the best split?

Key Insight

C4.5 can search for a threshold and convert a continuous feature into a useful decision rule.

Why Prune a Tree?

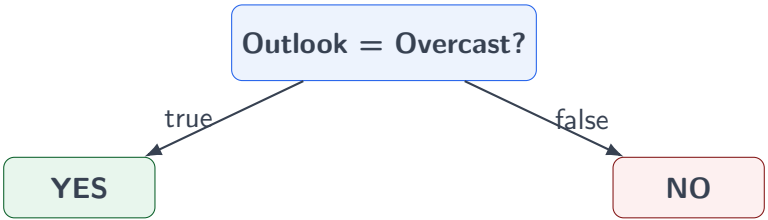


Too many branches $\rightarrow$ memorization

Pruning removes unnecessary complexity to improve generalization.

New Rule: Every Question Must Be YES / NO

What if every split had only two outcomes?



Key Insight

This is the basic split philosophy of CART: **binary splits only**.

CART: Classification and Regression Trees

Feature	CART
Split measure	Gini Index
Split type	Binary only
Classification	Yes
Regression	Yes
Pruning	Cost-complexity pruning

$$Gini(S) = 1 - \sum_i p_i^2$$

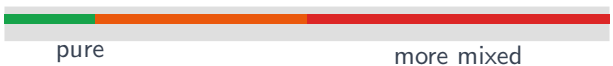
Gini: Same Idea, Different Measure

Pure node:

$$100\%\ \textit{Yes} \quad \Rightarrow \quad \textit{Gini} = 0$$

Mixed node:

$$50\%\ \textit{Yes},\ 50\%\ \textit{No} \quad \Rightarrow \quad \textit{Gini} = 0.5$$



Key Insight

Gini and entropy measure impurity differently, but both prefer purer child nodes.

CART: Find the Best Binary Split

For the Outlook feature, possible binary splits include:

Binary split	Weighted Gini
Sunny vs Others	0.394
Overcast vs Others	**0.357** $\star$
Rain vs Others	0.394

Decision

Lowest weighted Gini $\Rightarrow$ **Overcast vs Others**

Three Algorithms. Three Personalities.

ID3

"Reduce uncertainty"

Entropy
Information Gain

C4.5

"Be fair"

Gain Ratio
Pruning

CART

"Yes / No"

Gini
Binary splits

Quick Comparison

Feature	ID3	C4.5	CART
Split measure	Information Gain	Gain Ratio	Gini
Split type	Multi-way	Multi-way	Binary
Continuous attributes	No	Yes	Yes
Pruning	No	Yes	Yes
Regression	No	No	Yes

Challenge

If you were implementing a modern practical classifier, which properties would you want most? Why?

Now Let Python Build the Tree

We have done the hard conceptual work.



Let's see whether the computer agrees with our reasoning.

The Core Python Call

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=3,
    random_state=42
)

model.fit(X_train, y_train)
```

criterion='gini' $\rightarrow$ CART-style impurity

max_depth=3 $\rightarrow$ control tree complexity

Prediction Challenge

A new observation arrives:

Outlook	Sunny
Temperature	Hot
Humidity	High
Wind	Weak

Challenge

Predict: YES or NO?

Trace the path through the tree before running Python.

Visualize the Learned Tree

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(12,7))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=['No','Yes'],
    filled=True,
    rounded=True
)

plt.show()
```

Every node is a learned decision rule.

The Overfitting Experiment

Small tree
Generalizes

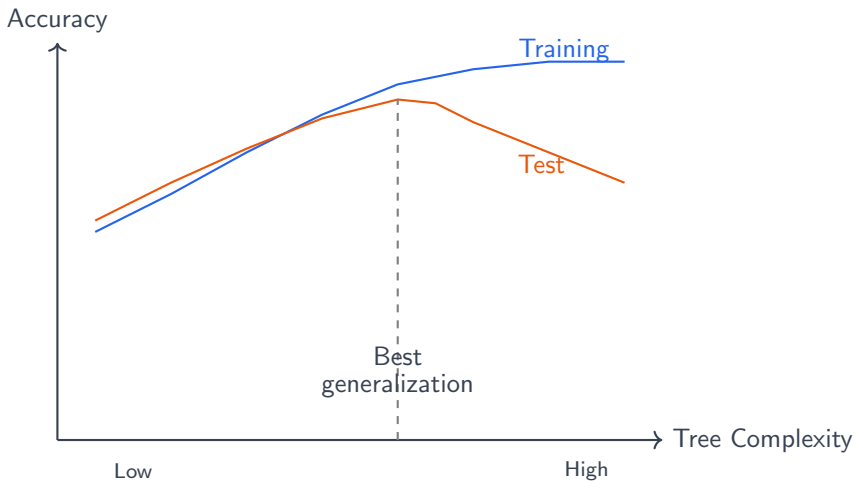
Very deep tree
Memorizes

Challenge

What happens to training accuracy and test accuracy as `max_depth` increases?

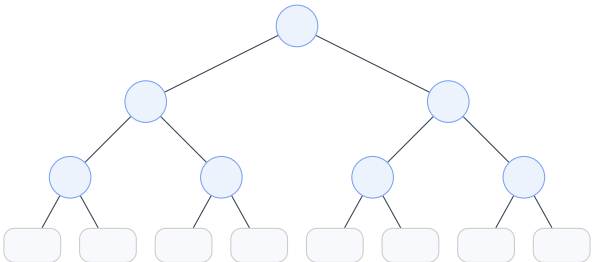
Run the experiment. Don't guess.

The Overfitting Curve



Why Do We Need Pruning?

A tree can become too clever.



The Problem

A very deep tree may memorize the training data instead of learning useful patterns.

Two Ways to Control a Tree

Pre-Pruning

Stop the tree from growing too much

`max_depth`
`min_samples_split`
`min_samples_leaf`

Post-Pruning

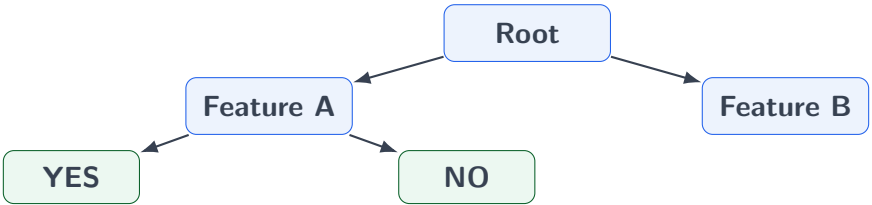
Grow first,
then remove unnecessary branches

Cost-complexity pruning

Simple intuition

Pre-pruning: "Don't grow too much."
Post-pruning: "Grow, then cut back."

Pre-Pruning: Stop the Tree Early

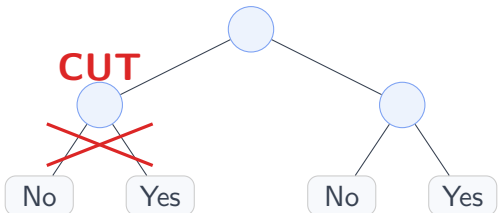


Instead of allowing unlimited growth, impose stopping rules.

Common parameters

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `max_leaf_nodes`

Post-Pruning: Grow First, Then Cut



If a subtree does not provide enough predictive benefit,
replace it with a simpler leaf.

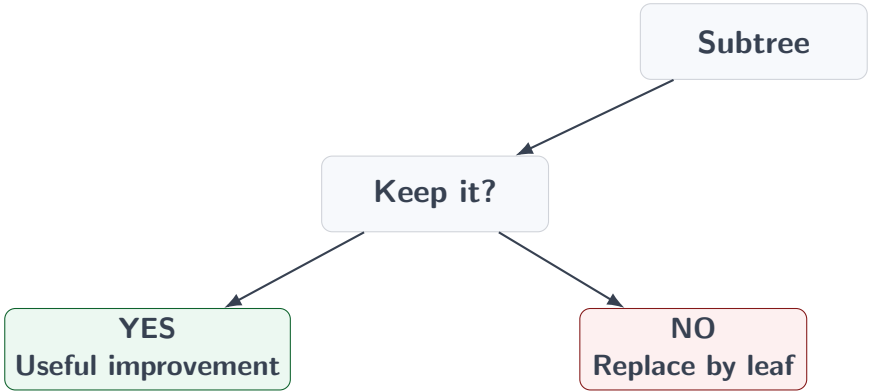
Controlling Tree Complexity

Parameter	What it controls
`max_depth`	Maximum tree depth
`min_samples_split`	Samples needed to split
`min_samples_leaf`	Minimum leaf size
`max_leaf_nodes`	Maximum number of leaves
`ccp_alpha`	Cost-complexity pruning

Rule of Thumb

A tree that perfectly memorizes training data may still be a poor model.

Should We Keep This Subtree?

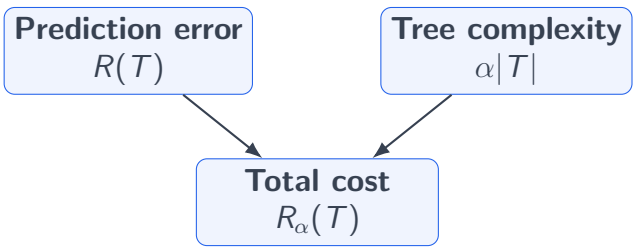


Pruning trades a little training fit for simpler, better-generalizing models.

CART: Cost-Complexity Pruning

What if we explicitly penalize complexity?

$$R_\alpha(T) = R(T) + \alpha|T|$$



Smaller tree + good predictive performance = better trade-off

Pruning in Python

```
from sklearn.tree import DecisionTreeClassifier
pruned_model = DecisionTreeClassifier(
    criterion='gini',
    ccp_alpha=0.01,
    random_state=42
)
pruned_model.fit(X_train, y_train)
```



Key Insight

Increasing ccp_alpha generally penalizes tree complexity more strongly, leading to a smaller tree.

Pruning Experiment: Which Tree Would You Deploy?

Model	Tree Size	Train Accuracy	Test Accuracy
Deep tree	25 leaves	100%	70%
Moderate tree	8 leaves	94%	86%
Pruned tree	5 leaves	91%	88%

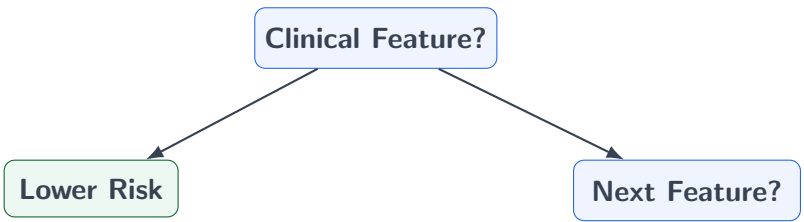
Challenge

Which one would you deploy?

Not necessarily the tree with the highest training accuracy.

From Tennis to Healthcare

Replace “Play Tennis?” with a clinical decision.



The attraction is the same: **human-readable decision paths**.

Can You Explain a Decision Tree in 60 Seconds?

Complete the chain:



Think

Expected ideas: choose a split → measure impurity/uncertainty → choose the best split → recurse until stopping/pruning.

Three Things to Remember

1. **ID3:** Entropy + Information Gain
2. **C4.5:** Gain Ratio + continuous attributes + pruning
3. **CART:** Gini + binary splitting + pruning + regression

The Big Idea

A decision tree keeps asking the most useful question.

Exit Ticket

Before you leave, answer these three questions:

1. Why is a pure node desirable?
2. Why can Information Gain be misleading?
3. Why does increasing tree depth sometimes hurt test performance?

Thank You

Questions?

